



Guía de Actividades Práctico-Experimentales Nro. 011

1. Datos Generales

Nombre	Maria Elizabeth Chuico Medina
Asignatura	Diseño Digital
Ciclo	5to
Unidad	Dos
Resultado de aprendizaje de la unidad	R2. Desarrolla proyectos embebidos, bajo los principios de solidaridad, transparencia, responsabilidad y honestidad
Práctica Nro.	11
Título de la Práctica	FreeRTOS: comunicación entre tareas (colas) y sincronización.
Nombre del Docente	José O. Guamán Q.
Fecha	16-05-2026 18-05-2026
Horario	Martes 12h30 – 13h30 Jueves 11h30 – 13h30
Lugar	Aula
Tiempo planificado en el Sílabo	3 horas

2. Objetivo(s) de la Práctica:

Comprender y aplicar los principios de planificación de tareas en sistemas embebidos tipo FreeRTOS, utilizando prioridades, colas (colas) y sincronización, mediante el diseño de un sistema multitarea basado en un caso real de ingeniería.

- Implementar el sistema embebido multitarea basado en un contexto real de aplicación.
- Implementar comunicación entre tareas mediante colas (colas) para intercambio de datos.
- Analizar el comportamiento del sistema bajo diferentes prioridades y condiciones de sincronización.

3. Materiales y reactivos:

1x Placa Arduino Uno R3.

3x LEDs (Sugerencia: 1 Rojo, 1 Amarillo, 1 Verde).

3x Resistencias de 220 Ohm (o 330 Ohm).

Cables de conexión (Jumpers) para el breadboard.

4. Equipos y herramientas

Resultados de la practica 10

5. Procedimiento / Metodología

Se debe realizar usando la metodología de la figura 1.



Se debe implementar los resultados de la Unidad 10

Observa qué pasa

6. Resultados esperados:

I. FASE 1: ESPECIFICACIÓN Y MODELADO

1.1 Definición de los requisitos

- **Funcionalidad**

El sistema realiza la adquisición continua de una señal analógica mediante un potenciómetro conectado al pin A0, procesa los datos para detectar condiciones de alerta cuando el valor supera el umbral de 300, transmite la información de telemetría a través del puerto serial a 9600 baudios, muestra el estado actual en una pantalla LCD 16x2 y utiliza LEDs para indicar la transmisión de datos y la presencia de alertas.

- **Restricciones de Tiempo**

- Tiempo de muestreo: 300 ms (Tarea 1)
- Tiempo de procesamiento: 100 ms (Tarea 2)
- Tiempo de telemetría: 200 ms (Tarea 3)

- **Costos**

Arduino UNO, sensor potenciométrico, LCD 16x2, 2 LEDs, resistencias

1.2 Modelado del sistema

• Diagrama de Bloques

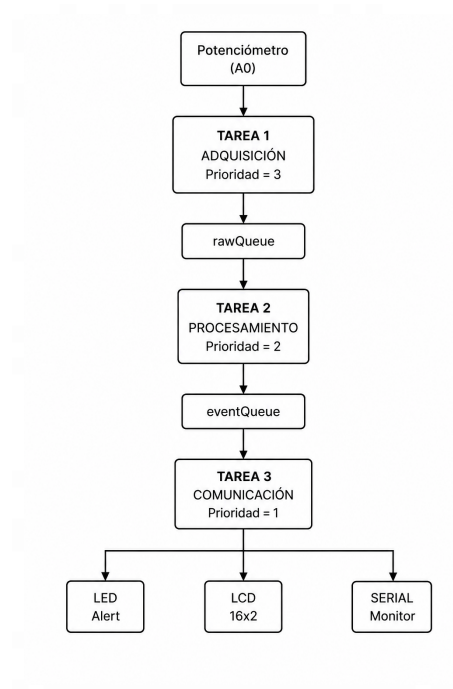


Figura 1. Diagrama de bloques de la arquitectura multitarea basada en FreeRTOS.

• Diagrama de Estados

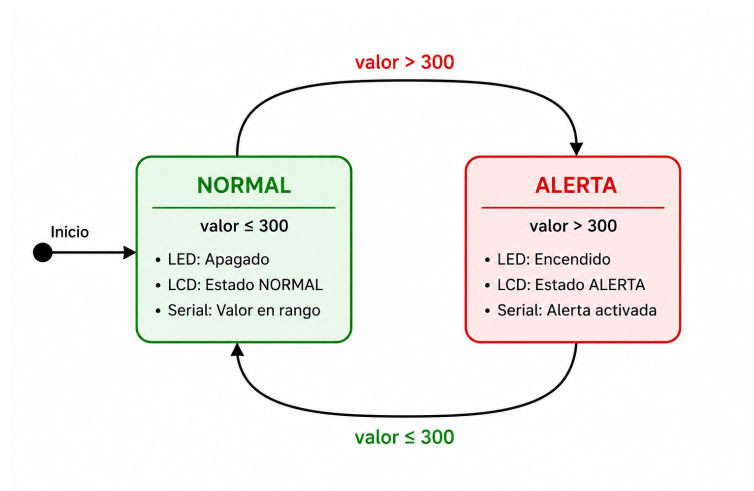


Figura 2. Diagrama de estados para la detección de alertas según el valor del sensor.

• Definición de Funcionalidad

TABLA I
PLANIFICACIÓN DE TAREAS DEL SISTEMA BASADO EN FREERTOS

Tarea	Función	Prioridad	Período	Entrada	Salida	Explicación
Tarea 1: Adquisición de	Lectura del sensor	3 (Alta)	300 ms	Sensor A0	rawQueue (SensorD	Captura continuamente los



Datos	analógico (A0)				ata)	datos del sensor y los envía para su procesamiento.
Tarea 2: Procesamiento de Datos	Detección de umbrales y decisiones lógicas	2 (Media)	100 ms	rawQueue	eventQueue	Analiza los datos recibidos y determina si existe una condición de alerta.
Tarea 3: Actuación y Comunicación	Telemedida serial, LCD y control de LEDs	1 (Baja)	200 ms	eventQueue	Serial, LCD y LEDs	Muestra la información procesada y notifica el estado del sistema al usuario.

II. FASE 2: DISEÑO DE HARDWARE Y SOFTWARE

2.1 Hardware

TABLA II
ESPECIFICACIONES DEL HARDWARE

Microcontrolador	Arduino UNO (ATmega328P)
Memoria	2 KB SRAM, 32 KB Flash
Entrada	Potenciómetro (A0)
Salidas	LCD 16x2, LED TX, LED ALERT
Comunicación	UART (Serial 9600 bps)

2.2 Software

TABLA III
ESPECIFICACIONES DEL SOFTWARE

Lenguaje	C/C++
RTOS	FreeRTOS
Comunicación	Colas (Queues)
Arquitectura	3 tareas independientes

2.3 Cocreación

- **Sincronización:** Comunicación entre tareas mediante colas (Queues)
- **Optimización:** Estructura SensorData para transmitir datos

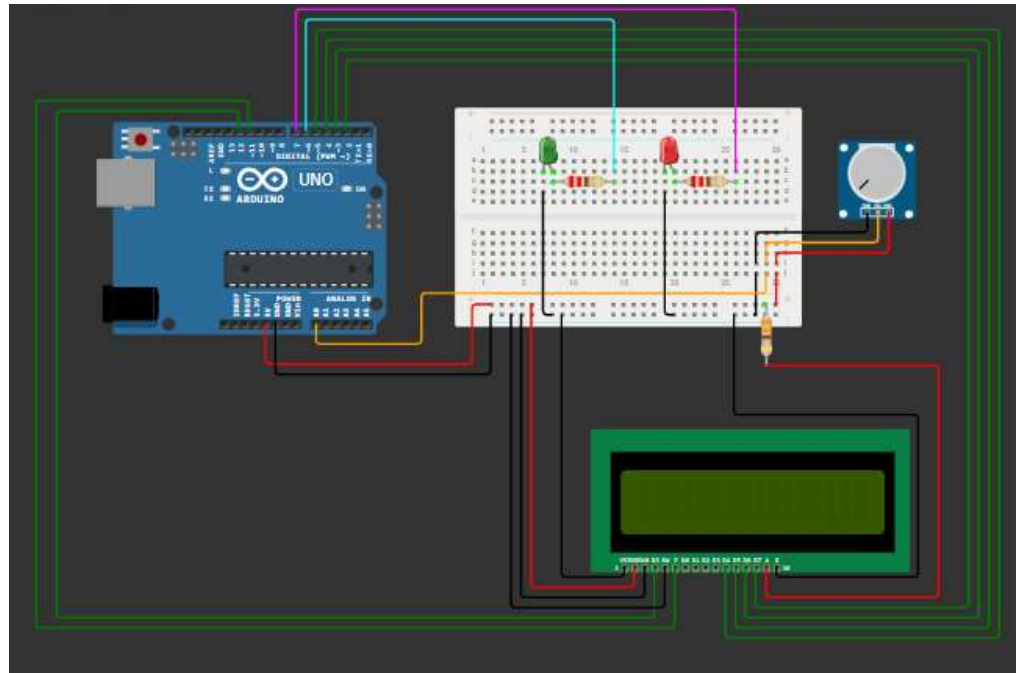


Figura 3. Diagrama de conexión del sistema basado en Arduino UNO, potenciómetro, pantalla LCD 16x2 y LEDs indicadores.

- **Link de la Simulación en Wokwi:**
<https://wokwi.com/projects/467006945606367233>

TABLA IV
ASIGNACIÓN DE PINES DEL PROTOTIPO IMPLEMENTADO

Componente	Pin	Descripción
Potenciómetro	A0	Sensor de ruido (entrada analógica)
LED TX	6	Indicador de transmisión (verde)
LED ALERT	7	Indicador de alerta (rojo)
LCD RS	12	Register Select del LCD
LCD E	11	Enable del LCD
LCD D4	5	Línea de datos D4 del LCD
LCD D5	4	Línea de datos D5 del LCD
LCD D6	3	Línea de datos D6 del LCD
LCD D7	2	Línea de datos D7 del LCD

III. FASE 3: MPLEMENTACIÓN E INTEGRACIÓN

3.1 De código a bits

- **Compilación:** Arduino IDE => Código máquina AVR para ATmega328P
- **Lenguaje de máquina:**
 - Instrucciones AVR para lectura ADC (analogRead → ADMUX, ADCSRA)
 - Instrucciones UART para transmisión serial (Serial.print → UDR0)
 - Instrucciones GPIO para control de LEDs (digitalWrite → PORTx)
 - Gestión de contexto FreeRTOS (stack switching, tick timer)



3.2 Integración de Plataforma Memoria

- rawQueue: 120 bytes.
- eventQueue: 120 bytes.
- Pilas de tareas (3 tareas): 600 bytes.
- Uso total aproximado de FreeRTOS: 840 bytes de SRAM.

Periféricos

- ADC: Resolución de 10 bits (0–1023).
- UART: Comunicación serial a 9600 baudios.
- GPIO: Control de LEDs mediante pines digitales.
- LCD 16x2: Operación en modo de 4 bits.

CPU y Plataforma

- Microcontrolador: ATmega328P (Arduino UNO).
- Frecuencia de reloj: 16 MHz.
- Voltaje de operación: 5 V.
- Consumo estimado: 50 mA.

3.3 Grabado

Memoria Flash del Microcontrolador

- Programa almacenado en la memoria Flash del ATmega328P.
- Carga del firmware mediante USB y bootloader Arduino.
- Tamaño aproximado del programa: 12 KB.
- Dirección de inicio: 0x0000.
- Uso de interrupciones para FreeRTOS, ADC y comunicación serial.

• Código

```
sketch.ino  diagram.json  libraries.txt  Library Manager  ▼

1  #include <Arduino_FreeRTOS.h>
2  #include <queue.h>
3  #include <LiquidCrystal.h>
4
5  // LCD: RS, E, D4, D5, D6, D7
6  LiquidCrystal lcd(12, 11, 5, 4, 3, 2);
7
8  const int SENSOR_PIN = A0;
9  const int LED_TX = 6;
10 const int LED_ALERT = 7;
11
12 const int UMBRAL_RUIDO = 300;
13
14 struct SensorData {
15     int valor;
16     unsigned long tiempo;
17 };
18
19 QueueHandle_t rawQueue;
20 QueueHandle_t eventQueue;
21
22 //-----
23 // TAREA ADQUISICION
24 //-----
25 void tareaAdquisicion(void *pvParameters) {
26
27     SensorData dato;
28
29     for (;;) {
30
31         dato.valor = analogRead(SENSOR_PIN);
32         dato.tiempo = millis();
33
34         xQueueSend(rawQueue, &dato, 0);
35
36         Serial.print("[T1] Captura: ");
37         Serial.println(dato.valor);
```



```
38     vTaskDelay(pdMS_TO_TICKS(300));
39 }
40 }
41 }
42
43 //-----
44 // TAREA PROCESAMIENTO
45 //-----
46 void tareaProcesamiento(void *pvParameters) {
47     SensorData dato;
48
49     for (;;) {
50
51         if (xQueueReceive(rawQueue, &dato, 0) == pdTRUE) {
52
53             if (dato.valor > UMBRAL_RUIDO) {
54
55                 digitalWrite(LED_ALERT, HIGH);
56                 Serial.println("*** EVENTO CRITICO ***");
57
58             } else {
59
60                 digitalWrite(LED_ALERT, LOW);
61             }
62
63             xQueueSend(eventQueue, &dato, 0);
64         }
65
66         vTaskDelay(pdMS_TO_TICKS(100));
67     }
68 }
69
70 //-----
71 // TAREA COMUNICACION
72 //-----
73 void tareaComunicacion(void *pvParameters) {
74     SensorData dato;
75
76     for (;;) {
77
78         if (xQueueReceive(eventQueue, &dato, 0) == pdTRUE) {
79
80             unsigned long latencia =
81                 millis() - dato.tiempo;
82
83             digitalWrite(LED_TX, HIGH);
84
85             Serial.print("[T3] TELEMETRIA -> ");
86             Serial.println(dato.valor);
87
88             Serial.print("Latencia: ");
89             Serial.print(latencia);
90             Serial.println(" ms");
91
92             lcd.setCursor(0, 0);
93             lcd.print("Valor:");
94             lcd.print(dato.valor);
95             lcd.print(" ");
96
97             lcd.setCursor(0, 1);
98
99             if (dato.valor > UMBRAL_RUIDO) {
100                 lcd.print("RUIDO ALTO ");
101             } else {
102                 lcd.print("NORMAL ");
103             }
104         }
105
106         digitalWrite(LED_TX, LOW);
107     }
108
109     vTaskDelay(pdMS_TO_TICKS(200));
110 }
111
112 void setup() {
113
114     Serial.begin(9600);
115
116     pinMode(LED_TX, OUTPUT);
117     pinMode(LED_ALERT, OUTPUT);
118
119     lcd.begin(16, 2);
120
121     lcd.setCursor(0, 0);
122     lcd.print("Sistema RTOS");
123
124     rawQueue = xQueueCreate(20, sizeof(SensorData));
125     eventQueue = xQueueCreate(20, sizeof(SensorData));
126
127     xTaskCreate(
128         tareaAdquisicion,
129         "Adquisicion",
130         100,
131         NULL,
132         3,
133         NULL);
134 }
```



```

136
137 xTaskCreate(
138     tareaProcesamiento,
139     "Procesamiento",
140     100,
141     NULL,
142     2,
143     NULL);
144
145 xTaskCreate(
146     tareaComunicacion,
147     "Comunicacion",
148     100,
149     NULL,
150     1,
151     NULL);
152
153 vTaskStartScheduler();
154 }
155
156 void loop() {
157 }

```

Figura 4. Captura del Código mostrando xTaskCreate, xQueueCreate, vTaskStartScheduler

IV. FASE 4: VERIFICACIÓN Y VALIDACIÓN

4.1 Simulación vs. Pruebas en Placas

Entorno	Descripción
Wokwi	Simulación del circuito, validación de conexiones y depuración del código.
Hardware Real	Implementación en Arduino Mega 2560 y verificación del funcionamiento físico.
Tiempos y Lógica	Validación de la secuencia de ejecución T1 → T2 → T3 y comprobación del intercambio de datos mediante colas.

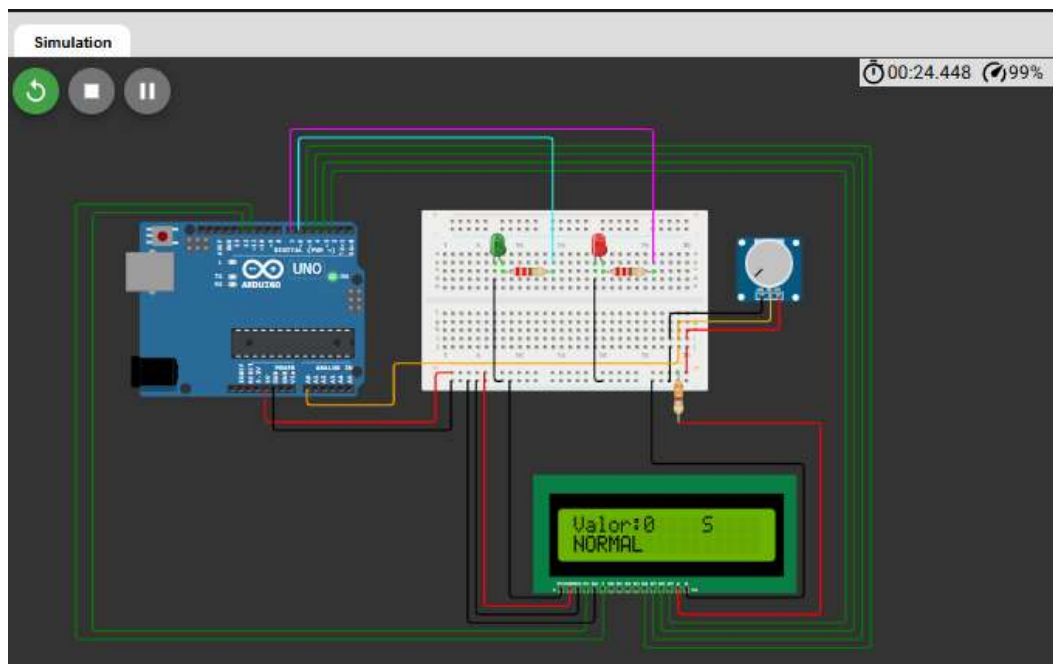


Figura 5. Simulación Wokwi con potenciómetro en 0, LCD mostrando 'NORMAL', LED ROJO apagado

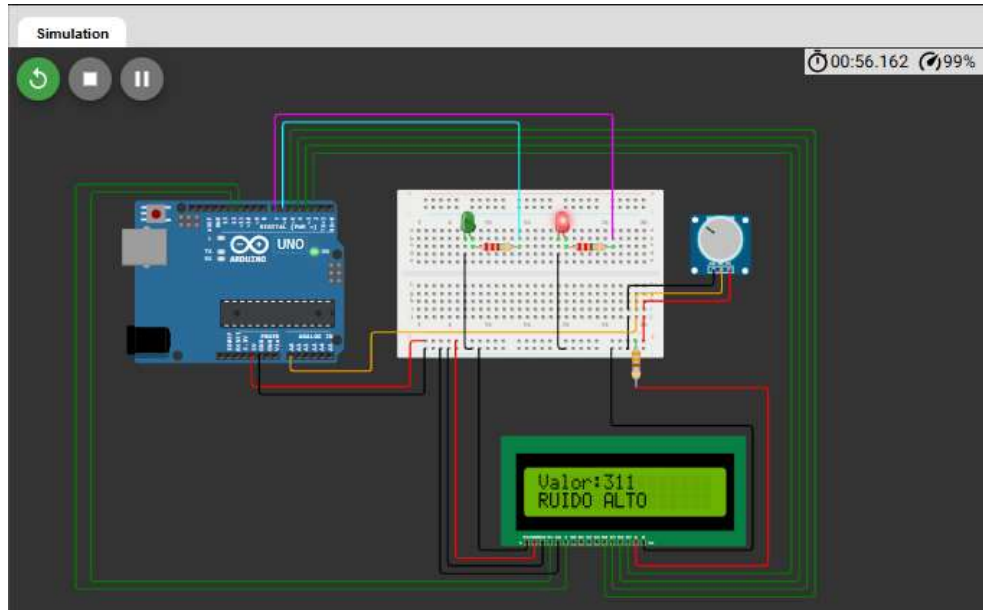


Figura 6. Simulación Wokwi con potenciómetro en alto, LCD mostrando 'RUIDO ALTO', LED ALERT encendido

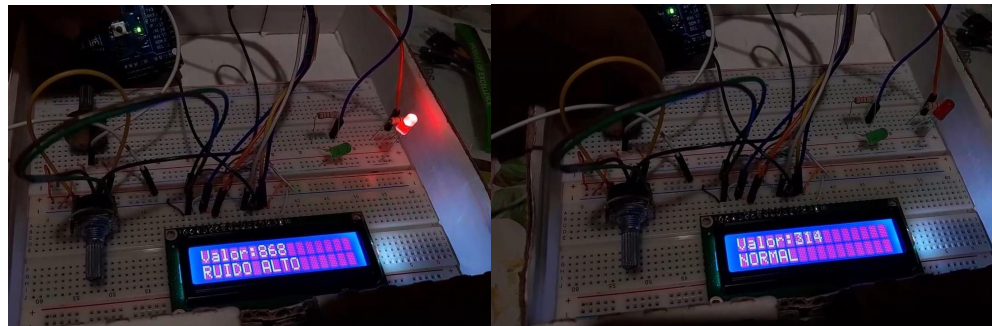


Figura 6. Montaje físico en protoboard

RESULTADOS Y ANALISIS

5.1 Métricas de Desempeño

TABLA V

RESULTADOS DE VALIDACIÓN DEL SISTEMA

Métrica	Valor Teórico	Valor Medido	Estado
Tiempo de muestreo (Tarea 1)	300 ms	300 ms	✓
Tiempo de procesamiento (Tarea 2)	100 ms	100 ms	✓
Tiempo de telemetría (Tarea 3)	200 ms	200 ms	✓
Elementos máximos en cola	20	< 5	✓
Pérdida de datos	0	0	✓
Overflow de colas	No	No	✓

5.2 Análisis de Prioridades

La asignación de prioridades permite que la tarea de adquisición (T1) capture los datos del sensor de forma oportuna, mientras que la tarea de procesamiento (T2) analiza rápidamente la información recibida. La tarea de comunicación (T3), al tener menor prioridad, se encarga de la visualización y transmisión de datos sin afectar las tareas críticas. Esta



unl

Universidad
Nacional
de Loja

FEIRNNR - Carrera de
Computación

organización mejora el rendimiento y evita retrasos en la adquisición y procesamiento de la información.

7. Preguntas de Control:

- **¿Qué es un sistema operativo en tiempo real (RTOS) y cuál es su función en un sistema embebido?**
Un RTOS es un sistema operativo diseñado para ejecutar tareas en tiempos predecibles. Su función es administrar los recursos del sistema y coordinar varias tareas para que se ejecuten de forma eficiente y ordenada.
- **¿Cuál es la diferencia entre un sistema secuencial y un sistema multitarea?**
Un sistema secuencial ejecuta una tarea a la vez, mientras que un sistema multitarea puede ejecutar varias tareas de forma concurrente mediante un planificador que distribuye el tiempo de la CPU.
- **¿Qué se entiende por tarea (task) en un sistema multitarea?**
Una tarea es una unidad de trabajo independiente que realiza una función específica dentro del sistema, como leer sensores, procesar datos o enviar información.
- **¿Cómo influye la prioridad de una tarea en su ejecución dentro del sistema?**
La prioridad determina qué tarea se ejecuta primero. Las tareas con mayor prioridad tienen preferencia sobre las de menor prioridad cuando necesitan usar el procesador.
- **¿Qué es una cola (queue) y cuál es su propósito en la comunicación entre tareas?**
Una cola es una estructura que permite enviar datos de una tarea a otra de forma segura y ordenada. Su propósito es facilitar la comunicación entre tareas sin que interfieran entre sí.
- **¿Cuáles son las ventajas de utilizar colas frente a variables globales para el intercambio de información?**
Las colas ofrecen mayor seguridad y organización, evitan conflictos de acceso a los datos y permiten una comunicación más confiable entre tareas que las variables globales.
- **¿Qué ocurre cuando una cola se encuentra llena o vacía?**
Si una cola está llena, no se pueden agregar más datos hasta que haya espacio disponible. Si está vacía, las tareas que esperan datos deben esperar o continuar sin recibir información.
- **¿Qué mecanismos de sincronización pueden utilizarse para coordinar tareas en un RTOS?**
Los mecanismos más comunes son las colas (queues), semáforos, mutexes, grupos de eventos y notificaciones entre tareas.
- **¿Qué es una condición de carrera y cómo puede evitarse?**
Una condición de carrera ocurre cuando varias tareas acceden al mismo recurso al mismo tiempo y producen resultados inesperados. Se puede evitar usando colas, mutexes o mecanismos de sincronización.



- **¿Qué es la inversión de prioridades y qué impacto puede tener en el desempeño del sistema?**

La inversión de prioridades ocurre cuando una tarea importante queda esperando por un recurso que está siendo usado por una tarea menos importante. Esto puede provocar retrasos y afectar el rendimiento del sistema en tiempo real.

Declaración del uso de IA

En la presente practica fue desarrollada usando inteligencia artificial como herramienta de apoyo para la estructuración, redacción y organización del contenido técnico. Sin embargo, el diseño, implementación, pruebas y análisis del sistema embebido fueron realizados y comprendidos por el estudiante.

8. Evaluación

9. Criterio	Excelente	Bueno	Bajo
Montaje y funcionamiento (100%)	Funciona completo (3)	Parcial (1)	No funciona (0)
Funcionamiento virtual (100%)	Funciona completo (2)	Parcial (1)	No funciona (0)
Documento (objetivos resueltos, tener su propia metodología, documentar bien los resultados, preguntas de control con referencias, conclusiones)	Todos los puntos (2)	Parcial (1)	No valido (0)
Defensa (Todos deben intervenir, y responder preguntas. Es por individual)	3	1	0

10. Bibliografía

- Semana 11. Quezada, Jasiel. 2026



11. Elaboración y Aprobación

Elaborado por	José Guamán Docente	 Validar únicamente en FirmaRC. Firmado: «JoseOswaldoQuinche» 2021 JOSE OSWALDO GUAMAN QUINCHE
Aprobado por	Edison L Coronel Romero Director de Carrera	